

**Laporan Tugas Besar 2**  
**IF3070 Dasar Intelegensi Artifisial**



**Disusun oleh :**

|                             |            |
|-----------------------------|------------|
| Muhammad Farhan             | - 18223004 |
| Jacob Reinhard M. Siagian   | - 18223026 |
| Stevan Einer Bonagabe       | - 18223028 |
| Hans Joseph B. W. Silitonga | - 18223072 |

**SISTEM DAN TEKNOLOGI INFORMASI**  
**SEKOLAH TEKNIK ELEKTRO DAN INFORMATIKA**  
**INSTITUT TEKNOLOGI BANDUNG**

## Daftar Isi

|                                                       |           |
|-------------------------------------------------------|-----------|
| <b>BAB I</b>                                          |           |
| <b>Data Cleaning dan Data Preprocessing</b>           | <b>3</b>  |
| 1.1 Data Cleaning                                     | 3         |
| 1.1.1 Penanganan Data Hilang (Handling Missing Data)  | 3         |
| 1.1.2 Dealing with Outliers                           | 5         |
| 1.1.3 Remove Duplicates                               | 6         |
| 1.1.4 Feature Engineering                             | 7         |
| 1.2 Data Preprocessing                                | 9         |
| 1.2.1 Feature Scaling                                 | 9         |
| 1.2.2 Feature Encoding                                | 11        |
| 1.2.3 Handling Imbalanced Dataset                     | 14        |
| 1.2.4 Dimensionality Reduction                        | 17        |
| 1.2.5 Data Normalization                              | 18        |
| 1.4 Preprocessing Pipeline                            | 19        |
| <b>BAB II</b>                                         |           |
| <b>Implementasi Decision Tree Learning</b>            | <b>22</b> |
| <b>BAB III</b>                                        |           |
| <b>Implementasi Logistic Regression</b>               | <b>33</b> |
| <b>BAB IV</b>                                         |           |
| <b>Perbandingan Hasil</b>                             | <b>39</b> |
| 4.1. Analisis Logistic Regression                     | 39        |
| Gambar 4.1 Hasil Perbandingan performa LogReg Scratch | 39        |
| Gambar 4.2 Hasil Perbandingan performa LogReg Scratch | 40        |
| Gambar 4.3 Hasil Perbandingan performa LogReg Pustaka | 41        |
| 4.2. Analisis Decision Tree Learning                  | 42        |
| Gambar 4.4 Hasil performa DTL Scratch                 | 43        |
| Gambar 4.5 Hasil performa DTL Scratch                 | 44        |
| Gambar 4.5 Hasil performa DTL Pustaka                 | 45        |
| Gambar 4.6 Hasil performa DTL Pustaka                 | 46        |
| <b>BAB V</b>                                          |           |
| <b>Akhir &amp; Referensi</b>                          | <b>47</b> |
| 5.1 Pembagian Tugas                                   | 47        |
| 5.2 Referensi                                         | 47        |

# BAB I

## Data Cleaning dan Data Preprocessing

### 1.1 Data Cleaning

#### 1.1.1 Penanganan Data Hilang (Handling Missing Data)

```
# FUNGSI: IMPUTASI (MISSING VALUES)

def impute_missing_values(data):

    data = data.copy()

    # Identifikasi kolom numerik dan kategorikal

    kolom_numerik =
data.select_dtypes(include=['number']).columns.tolist()

    # Buang target jika ada

    if 'is_fraud' in kolom_numerik:

        kolom_numerik.remove('is_fraud')

    kolom_categorikal =
data.select_dtypes(include=['object']).columns.tolist()

    print(f"Imputasi - Kolom Numerik: {len(kolom_numerik)},
Kategorikal: {len(kolom_categorikal)}")

    # Imputasi Numerik dengan MEDIAN

    for kolom in kolom_numerik:

        if kolom in data.columns:

            median_val = data[kolom].median()
```

```

        data[kolom] = data[kolom].fillna(median_val)

# Imputasi Kategorikal dengan MODUS
for kolom in kolom_categorikal:

    if kolom in data.columns:

        if not data[kolom].mode().empty:

            mode_val = data[kolom].mode().iloc[0]

            data[kolom] = data[kolom].fillna(mode_val)

        else:

            data[kolom] = data[kolom].fillna("Unknown")

    print(f"Sisa NaN setelah imputasi:
{data.isnull().sum().sum()}")

return data

```

**Metode yang Dipilih adalah** Imputasi Median untuk Data Numerik dan Modus untuk Data Kategorikal. Alasan kami memilih metode ini adalah

1. **Fitur Numerik (Imputasi Median):** Kami memilih menggunakan nilai Median daripada Rata-rata (Mean) untuk mengisi kekosongan pada data numerik, seperti “transaction\_amount” atau “avg\_transaction\_amount”. Hal ini dikarenakan dataset transaksi keuangan cenderung memiliki distribusi yang skewed dan mengandung outliers yang ekstrim. Penggunaan Mean akan sangat sensitif terhadap nilai ekstrim sehingga dapat membiaskan representasi data, sedangkan Median lebih “robust” dalam merepresentasikan nilai tengah yang sebenarnya dari populasi data.
2. **Fitur Kategorikal (Imputasi Modus):** Untuk fitur non-numerik, kami menggunakan nilai Modus. Ini adalah pendekatan standar yang efektif untuk menjaga konsistensi distribusi kategori dalam dataset tanpa memperkenalkan kategori baru yang artifisial.

Kami tidak memilih untuk menghapus baris data karena berisiko menghilangkan sampel kelas minoritas (fraud) yang penting bagi proses pembelajaran model.

### 1.1.2 Dealing with Outliers

```
# FUNGSI: HANDLING OUTLIERS

def clip_outliers(data, lower_percentile=0.01,
upper_percentile=0.99):

    data = data.copy()

    # Identifikasi kolom numerik

    kolom_numeric =
data.select_dtypes(include=['number']).columns.tolist()

    if 'is_fraud' in kolom_numeric:

        kolom_numeric.remove('is_fraud')

    print(f"Clipping {len(kolom_numeric)} kolom (Persentil
{lower_percentile*100}%-{upper_percentile*100}%)...")

    for kolom in kolom_numeric:

        if kolom in data.columns:

            lower_limit = data[kolom].quantile(lower_percentile)

            upper_limit = data[kolom].quantile(upper_percentile)

            data[kolom] = data[kolom].clip(lower_limit,
upper_limit)

    print("✓ Clipping selesai")
```

```
return data
```

**Metode yang Dipilih:** *Clipping* pada persentil 1% dan 99%.

Alasan kami memilih metode ini karena dalam dataset *fraud detection*, data pencilan (*outliers*) sering kali merepresentasikan perilaku anomali yang menjadi indikator utama fraud (misalnya, nominal transaksi yang sangat besar). Oleh karena itu, kami memutuskan untuk **tidak menghapus** data outlier tersebut karena berpotensi menghilangkan informasi krusial.

Metode ini membatasi nilai fitur pada batas bawah (persentil ke-1) dan batas atas (persentil ke-99). Nilai yang berada di luar rentang ini akan diganti dengan nilai batas terdekat (misalnya, nilai di atas persentil 99 akan di set sama dengan nilai persentil 99). Pendekatan ini memungkinkan model untuk tetap mengenali data tersebut sebagai nilai yang "sangat tinggi" atau "sangat rendah", namun mencegah nilai yang terlalu ekstrim dan dapat merusak stabilitas perhitungan model.

Untuk menjaga integritas evaluasi model, batas persentil (*threshold*) dihitung hanya menggunakan Training Set. Lalu nilai batas diterapkan pada Validation Set dan Test Set untuk mencegah kebocoran data.

### 1.1.3 Remove Duplicates

```
# FUNGSI: PENGECEKAN DUPLIKAT (3 LAPIS)

def remove_duplicates(data):
    data = data.copy()
    initial_rows = data.shape[0]

    print(f"Jumlah baris awal: {initial_rows}")

    # LAPIS 1: Cek ID
    if 'ID' in data.columns:
        dup_id = data.duplicated(subset=['ID']).sum()
        if dup_id > 0:
            print(f"ID duplikat: {dup_id}")
            data = data.drop_duplicates(subset=['ID'],
keep='first')

            data = data.drop(columns=['ID'], errors='ignore')
            print("ID diperiksa & dihapus")

    # LAPIS 2: Cek transaction_id
```

```

    if 'transaction_id' in data.columns:
        dup_tid =
data.duplicated(subset=['transaction_id']).sum()
        if dup_tid > 0:
            print(f"Transaction_id duplikat: {dup_tid}")
            data =
data.drop_duplicates(subset=['transaction_id'], keep='first')

            data = data.drop(columns=['transaction_id'],
errors='ignore')
            print("Transaction_id diperiksa & dihapus")

# LAPIS 3: Cek konten duplikat
dup_content = data.duplicated().sum()
if dup_content > 0:
    print(f"[Lapis 3] Konten duplikat: {dup_content}")
    data = data.drop_duplicates(keep='first')
else:
    print("Tidak ada duplikat konten")

final_rows = data.shape[0]
print(f" Total dihapus: {initial_rows - final_rows}")

return data

```

**Metode yang Dipilih:** Multi-Stage Deduplication. Alasan kami memiliki metode ini adalah:

1. **Pemeriksaan Primary Key (ID & transaction\_id):** Pertama, kami memeriksa duplikasi berdasarkan kolom identitas unik. Karena setiap baris dan transaksi seharusnya memiliki ID yang unik, kemunculan ganda pada kolom ini mengindikasikan kesalahan sistem atau pencatatan ganda. Baris dengan ID ganda dihapus, kemudian kolom ID dan transaction\_id itu sendiri dibuang dari dataset karena tidak berisi pola prediktif yang berguna.
2. **Pemeriksaan Konten (*Content-Based Deduplication*):** Setelah kolom identitas dihapus, kami melakukan pemeriksaan ulang terhadap isi data. Langkah ini bertujuan menangkap kasus di mana transaksi yang identik (misalnya: *user*, waktu, dan nominal yang sama persis) tercatat lebih dari satu kali namun dengan ID yang berbeda.

#### 1.1.4 Feature Engineering

```

def create_features(data):
    data = data.copy()

```

```

print("Membuat Interaction Features...")
# Rasio dan gabungan
data['amount_ratio'] = data['transaction_amount'] / (data['avg_transaction_amount'] + 0.001)
data['velocity_ratio'] = data['transactions_last_1h'] / (data['transactions_last_24h'] + 0.001)
data['total_external_risk'] = data['ip_risk_score'] + data['merchant_risk'] +
data['country_risk']
data['device_security_flag'] = (data['ip_risk_score'] + (1 - data['device_trust_score'])) / 2

print("Membuat Binning Features...")
# Age Group
labels_age = ['Young', 'Adult', 'Senior']
bins_age = [0, 25, 60, 150]
data['age_group'] = pd.cut(data['age'], bins=bins_age, labels=labels_age)

# Time Periode
labels_time = ['Night', 'Morning', 'Afternoon', 'Evening']
bins_time = [-1, 6, 12, 18, 24]
data['time_period'] = pd.cut(data['time_of_day'], bins=bins_time, labels=labels_time)

print("Membuat Behavioral Features...")
# Behavioral anomaly
data['suspicious_login'] = (data['failed_login_attempts'] > 2).astype(int)
data['is_weekend'] = data['day_of_week'].isin([5, 6]).astype(int)

print("Feature Engineering selesai")

return data

```

**Metode yang Dipilih:** Kombinasi *Interaction Features*, *Binning*, dan *Domain-Specific Indicators*. Alasan kami memilih metode ini:

1. **Interaction Features (Fitur Interaksi & Rasio):** Kami menciptakan fitur baru yang menggambarkan hubungan antar variabel. Contohnya `amount_ratio` (rasio nilai transaksi saat ini terhadap rata-rata historis pengguna) dan `velocity_ratio` (proporsi aktivitas transaksi dalam 1 jam terakhir dibandingkan total 24 jam). Fitur rasio ini jauh lebih informatif daripada nilai nominal karena dapat mendeteksi anomali perilaku yang spesifik untuk setiap pengguna. Selain itu, kami menggabungkan berbagai skor risiko terpisah (`ip_risk`, `merchant_risk`, `country_risk`) menjadi satu indikator komprehensif `total_external_risk` untuk memberikan sinyal risiko yang lebih kuat kepada model.
2. **Binning / Discretization (Pengelompokan):** Fitur kontinu seperti `age` dan `time_of_day` dikelompokkan menjadi kategori diskrit (`age_group` dan `time_period`). Hal ini bertujuan untuk mengurangi *noise* pada data numerik dan membantu model menangkap pola non-linear yang bersifat umum, seperti kecenderungan fraud pada kelompok usia tertentu (misal: Lansia) atau waktu kejadian tertentu (misal: Tengah Malam).
3. **Domain-Specific Behavioral Indicators (Indikator Perilaku):** Berdasarkan pengetahuan domain (*domain knowledge*), kami menambahkan penanda biner untuk



perilaku yang mencurigakan. Fitur `suspicious_login` dibuat untuk menandai upaya pengambilalihan akun (*account takeover*), di mana transaksi didahului oleh kegagalan login berulang kali. Kami juga menambahkan fitur `is_weekend` karena pola fraud seringkali mengeksploitasi waktu di luar jam kerja perbankan standar.

## 1.2 Data Preprocessing

Bagian ini menjelaskan rangkaian data preprocessing yang diterapkan sebelum pemodelan. Tujuan utamanya adalah menyiapkan data agar lebih konsisten, informatif, dan sesuai dengan asumsi algoritma yang akan digunakan. Secara garis besar, langkah-langkah mencakup penskalaan fitur numerik, pengkodean fitur kategorikal, penanganan ketidakseimbangan kelas, normalisasi, serta reduksi dimensi. Setiap langkah dipilih berdasarkan karakteristik data (misalnya adanya outlier, distribusi yang miring, banyaknya kategori unik), serta dampak yang diharapkan terhadap proses pelatihan dan evaluasi model. Dengan susunan yang rapi dan disiplin pemisahan “fit di data latih” serta “transform di data validasi/uji”, proses ini membantu menghindari kebocoran data dan meningkatkan keandalan hasil.

### 1.2.1 Feature Scaling

```
from sklearn.base import BaseEstimator, TransformerMixin
from sklearn.preprocessing import MinMaxScaler, StandardScaler, RobustScaler
import numpy as np

class FeatureScaler(BaseEstimator, TransformerMixin):
    def __init__(self, cols_to_scale, scaler_type='minmax'):
        """
        Custom Scaler :
        1. MinMax (Normalization) -> Bagus untuk KNN
        2. Standard (Standardization) -> Bagus untuk Logistic Regression
        3. Robust -> Bagus jika banyak Outlier
        4. Log -> Bagus untuk data skewed timpang seperti Transaction Amount

        Parameters:
        -----
        cols_to_scale : list
            Daftar nama kolom yang ingin di-scaling.
        scaler_type : str
            Pilihan: 'minmax', 'standard', 'robust', atau 'log'.
```

```

"""
self.cols_to_scale = cols_to_scale
self.scaler_type = scaler_type
self.scaler = None

def fit(self, X, y=None):
    # Log Transformation tidak butuh fit karna nggak perlu menghitung
rata-rata dulu
    if self.scaler_type == 'log':
        return self

    # Memilih scaler untuk 3 metode lainnya
    if self.scaler_type == 'minmax':
        self.scaler = MinMaxScaler()
    elif self.scaler_type == 'standard':
        self.scaler = StandardScaler()
    elif self.scaler_type == 'robust':
        self.scaler = RobustScaler()
    else:
        # Default fallback
        self.scaler = MinMaxScaler()

    # Fit hanya pada kolom yang dipilih
    if self.cols_to_scale:
        self.scaler.fit(X[self.cols_to_scale])

    return self

def transform(self, X):
    X_scaled = X.copy()

    # Pastikan ada kolom yang mau diubah
    if self.cols_to_scale:

        # Khusus Log Transformation
        if self.scaler_type == 'log':
            # Menggunakan np.log1p (log(1+x)) biar aman kalau ada angka 0
            # terapkan hanya ke kolom yang dipilih

```

```

        X_scaled[self.cols_to_scale] =
np.log1p(X_scaled[self.cols_to_scale])

        # Logic untuk Scaler biasa (MinMax, Standard, Robust)
        else:
            X_scaled[self.cols_to_scale] =
self.scaler.transform(X[self.cols_to_scale])

    return X_scaled

```

Dalam dataset ini, kami menghadapi masalah distribusi yang sangat tidak merata pada beberapa fitur numerik, terutama `transaction_amount` dan `distance_from_home`. Untuk mengatasi hal ini, kami menerapkan dua tahap penskalaan secara berurutan:

Pertama, kami menggunakan log transformation dengan fungsi `log1p` pada kedua fitur tersebut, karena kedua fitur ini memiliki nilai yang sangat timpang (skewed), di mana sebagian besar transaksi bernilai kecil tetapi ada beberapa yang sangat besar. Transformasi logaritmik membantu "meratakan" distribusi ini sehingga model tidak terlalu bias terhadap nilai ekstrim.

Kedua, untuk seluruh fitur numerik lainnya (seperti `age`, `time_of_day`, `num_prev_transactions`, `device_trust_score`, dan lain-lain), kami menggunakan `RobustScaler`, karena dataset ini kemungkinan besar masih memiliki outlier walaupun sudah dilakukan clipping sebelumnya. `RobustScaler` lebih tahan terhadap outlier dibandingkan `StandardScaler` karena menggunakan median dan IQR (Interquartile Range). Ini penting agar fitur-fitur tetap memiliki skala yang konsisten tanpa terdistorsi oleh nilai ekstrim.

### ***1.2.2 Feature Encoding***

```

from sklearn.base import BaseEstimator, TransformerMixin
from sklearn.preprocessing import OrdinalEncoder, OneHotEncoder
import pandas as pd
import numpy as np

class FeatureEncoder(BaseEstimator, TransformerMixin):
    def __init__(self, cols_to_encode, encoding_type='onehot'):
        """
        Custom Encoder :
        1. label      : Mengubah kategori jadi angka urut (0, 1, 2...)

```

- 2. onehot : Membuat kolom biner 0/1 untuk tiap kategori
- 3. target : Mengubah kategori jadi rata-rata target (Mean Encoding)

Parameters:

-----

cols\_to\_encode : list

Daftar kolom yang akan di-encode.

encoding\_type : str

Pilihan: 'label', 'onehot', atau 'target'.

"""

self.cols\_to\_encode = cols\_to\_encode

self.encoding\_type = encoding\_type

# Variabel untuk menyimpan kamus encoding

self.encoder = None

self.target\_means = {} # Khusus untuk Target Encoding

self.global\_mean = 0 # Default jika ada kategori baru di target

encoding

def fit(self, X, y=None):

# 1. Label Encoding

if self.encoding\_type == 'label':

# Kita gunakan OrdinalEncoder dari sklearn (fungsinya sama dengan Label Encoding untuk fitur)

# handle\_unknown='use\_encoded\_value' agar aman jika ada data asing di test set

self.encoder = OrdinalEncoder(handle\_unknown='use\_encoded\_value', unknown\_value=-1)

self.encoder.fit(X[self.cols\_to\_encode])

# 2. One-Hot Encoding

elif self.encoding\_type == 'onehot':

self.encoder = OneHotEncoder(sparse\_output=False, handle\_unknown='ignore')

self.encoder.fit(X[self.cols\_to\_encode])

# 3. Target Encoding (Mean Encoding)

elif self.encoding\_type == 'target':

```

        if y is None:
            raise ValueError("Metode 'target' encoding WAJIB menyertakan
parameter y (label) saat fit!")

        # Hitung rata-rata global (jaga-jaga untuk data baru)
        self.global_mean = y.mean()

        # Gabungkan X dan y sementara untuk hitung rata-rata
        temp_df = X.copy()
        temp_df['target'] = y

        for col in self.cols_to_encode:
            # Hitung rata-rata target per kategori
            means = temp_df.groupby(col)['target'].mean()
            self.target_means[col] = means

    return self

def transform(self, X):
    X_encoded = X.copy()

    # 1. Proses Label Encoding
    if self.encoding_type == 'label':
        X_encoded[self.cols_to_encode] =
self.encoder.transform(X[self.cols_to_encode])

    # 2. Proses One-Hot Encoding
    elif self.encoding_type == 'onehot':
        ohe_results = self.encoder.transform(X[self.cols_to_encode])
        feature_names =
self.encoder.get_feature_names_out(self.cols_to_encode)
        ohe_df = pd.DataFrame(ohe_results, columns=feature_names,
index=X.index)

        # Gabung dan buang kolom lama
        X_encoded = pd.concat([X_encoded, ohe_df], axis=1)
        X_encoded.drop(columns=self.cols_to_encode, inplace=True)

```

```

# 3. Proses Target Encoding
elif self.encoding_type == 'target':
    for col in self.cols_to_encode:
        # Map nilai kategori ke angka rata-ratanya
        # Jika ada kategori baru (NaN), isi dengan rata-rata global
        X_encoded[col] =
X_encoded[col].map(self.target_means[col]).fillna(self.global_mean)

    return X_encoded

```

Dataset memiliki banyak fitur kategorikal dengan karakteristik yang berbeda-beda. Kami membaginya menjadi dua kelompok, yakni:

Kelompok pertama adalah fitur dengan sedikit kategori unik (low cardinality), seperti gender, device\_type, transaction\_type, day\_of\_week, dan is\_new\_country. Untuk fitur-fitur ini, kami menggunakan one-hot encoding. Alasannya karena nilai-nilai tersebut bersifat nominal (tidak ada urutan alamiah), dan jumlahnya tidak banyak sehingga tidak akan menyebabkan ledakan dimensi (dimensionality explosion).

Kelompok kedua adalah fitur dengan banyak kategori unik (high cardinality), seperti merchant\_category, country, dan device\_os. Pada fitur ini, kami menggunakan target encoding (mean encoding), yaitu mengganti setiap kategori dengan rata-rata nilai target (is\_fraud) untuk kategori tersebut berdasarkan data latih. Teknik ini tetap mempertahankan informasi prediktif sambil menjaga dimensi tetap terkendali.

Perlu diperhatikan bahwa target encoding hanya dilakukan pada data latih (fit pada X\_train dan y\_train), lalu statistiknya diterapkan pada data validasi dan test (transform saja). Jika ada kategori baru yang muncul saat inferensi (tidak ada di data latih), sistem akan menggunakan rata-rata global sebagai nilai fallback untuk menghindari error.

### ***1.2.3 Handling Imbalanced Dataset***

```

from sklearn.base import BaseEstimator, TransformerMixin
from sklearn.utils import resample
from imblearn.over_sampling import SMOTE
import pandas as pd
import numpy as np

class ImbalanceHandler(BaseEstimator, TransformerMixin):

```

```

def __init__(self, method='smote', random_state=42):
    """
    Class untuk menangani Imbalance Dataset:
    - 'smote' -> Generate synthetic samples (recommended)
    - 'random_oversampling' -> Duplicate minority class
    - 'random_undersampling' -> Downsample majority class
    """
    self.method = method
    self.random_state = random_state

def fit(self, X, y=None):
    return self

def transform(self, X, y):
    # 1. Jika method SMOTE, gunakan library imblearn langsung
    # Ini handle input baik berupa DataFrame maupun Numpy Array
    if self.method == 'smote':
        smote = SMOTE(random_state=self.random_state)
        X_res, y_res = smote.fit_resample(X, y)
        return X_res, y_res

    # 2. Jika method Manual (Random Over/Under Sampling)
    # Kita perlu menggabungkan X dan y sementara
    if isinstance(X, pd.DataFrame):
        data = X.copy()
    else:
        # Jika input dari Pipeline berupa Numpy Array (misal hasil PCA),
        ubah ke DF
        data = pd.DataFrame(X)

    # Pastikan y berupa array untuk assignment
    y_vals = y.values if isinstance(y, pd.Series) else y
    data['target_temp_label'] = y_vals

    majority = data[data['target_temp_label'] == 0]
    minority = data[data['target_temp_label'] == 1]

    if self.method == 'random_oversampling':

```

```

        minority_upsampled = resample(
            minority,
            replace=True,
            n_samples=len(majority),
            random_state=self.random_state
        )
        data_resampled = pd.concat([majority, minority_upsampled])

    elif self.method == 'random_undersampling':
        majority_downsampled = resample(
            majority,
            replace=False,
            n_samples=len(minority),
            random_state=self.random_state
        )
        data_resampled = pd.concat([majority_downsampled, minority])

    else:
        return X, y

    # Pisahkan kembali
    y_resampled = data_resampled['target_temp_label']
    X_resampled = data_resampled.drop('target_temp_label', axis=1)

    # Kembalikan ke format numpy jika input awalnya numpy
    if not isinstance(X, pd.DataFrame):
        return X_resampled.values, y_resampled.values

    return X_resampled, y_resampled

print("Class ImbalanceHandler sudah ada")

```

Untuk mengatasi ketidakseimbangan kelas yang ekstrim, seperti jumlah transaksi normal jauh lebih banyak daripada transaksi fraud, kami menggunakan SMOTE (Synthetic Minority Over-sampling Technique) setelah seluruh pipeline preprocessing selesai. SMOTE membangkitkan sampel sintetis kelas minoritas (fraud) dengan cara interpolasi antara sampel-sampel minoritas yang ada, sehingga distribusi kelas menjadi lebih seimbang.



Namun, SMOTE hanya diterapkan pada data latih, tidak pada data validasi atau test. Karena data validasi dan test harus tetap mencerminkan kondisi real-world yang tidak seimbang. Jika kami juga meng-SMOTE data validasi, evaluasi performa model tidak akan objektif dan bisa menyebabkan overfitting pada kelas fraud.

#### ***1.2.4 Dimensionality Reduction***

```
# Dimensionality Reduction (PCA) hanya definisi kelas
from sklearn.base import BaseEstimator, TransformerMixin
from sklearn.decomposition import PCA

class DimensionalityReducer(BaseEstimator, TransformerMixin):
    def __init__(self, n_components=None):
        """
        Reduksi dimensi menggunakan PCA.
        n_components:
            - None : pass-through (tidak mengubah data)
            - int  : jumlah komponen PCA
            - float : proporsi varian yang dipertahankan (mis. 0.95)
        """
        self.n_components = n_components
        self.pca = None

    def fit(self, X, y=None):
        if self.n_components is None:
            return self
        self.pca = PCA(n_components=self.n_components)
        self.pca.fit(X)
        return self

    def transform(self, X):
        if self.n_components is None or self.pca is None:
            return X
        return self.pca.transform(X)
```

Setelah proses encoding, jumlah fitur bertambah cukup signifikan, terutama dari one-hot encoding. Untuk mengurangi kompleksitas dan biaya komputasi tanpa kehilangan informasi

penting, kami menerapkan PCA (Principal Component Analysis) dengan mempertahankan 95% varian total data (`n_components=0.95`).

PCA bekerja dengan membuat kombinasi linear dari fitur-fitur asli menjadi "komponen utama" (principal components) yang tidak berkorelasi. Ini memiliki dua manfaat utama, yakni mengurangi dimensi (Model lebih cepat dilatih dan tidak mudah overfit) dan mengatasi multikolinearitas, dimana Fitur-fitur yang berkorelasi tinggi (misalnya hasil one-hot encoding) dikombinasikan menjadi komponen yang independen.

Dengan threshold 95%, kami yakin bahwa informasi penting tetap terjaga sambil memangkas fitur-fitur yang kurang berkontribusi.

### ***1.2.5 Data Normalization***

```
from sklearn.base import BaseEstimator, TransformerMixin
from sklearn.preprocessing import StandardScaler

class DataNormalizer(BaseEstimator, TransformerMixin):
    def __init__(self, cols_to_normalize):
        """
        Class untuk Data Normalization (Standardization / Z-Score).
        Tujuannya: Membuat distribusi data memiliki Mean=0 dan Std=1.

        Parameters:
        -----
        cols_to_normalize : list
            Daftar nama kolom numerik yang akan di-standarisasi.
        """
        self.cols_to_normalize = cols_to_normalize
        self.scaler = StandardScaler()

    def fit(self, X, y=None):
        # Mesin belajar Rata-rata (Mean) dan Standar Deviasi dari data train
        if self.cols_to_normalize:
            self.scaler.fit(X[self.cols_to_normalize])
        return self

    def transform(self, X):
        X_norm = X.copy()
```

```
# Terapkan rumus Z-Score:  $z = (x - \text{mean}) / \text{std\_dev}$ 
if self.cols_to_normalize:
    X_norm[self.cols_to_normalize] =
self.scaler.transform(X[self.cols_to_normalize])

return X_norm
```

Dalam kode, kami mendefinisikan kelas `DataNormalizer` yang menggunakan `StandardScaler` (Z-score normalization) untuk membuat distribusi fitur memiliki mean=0 dan standard deviation=1. Namun, kelas ini tidak digunakan dalam pipeline aktif karena kami sudah menggunakan `RobustScaler` yang lebih sesuai untuk data dengan outlier.

## 1.4 Preprocessing Pipeline

Seluruh tahapan di atas disusun secara berurutan dalam satu Pipeline dari scikit-learn. Urutan ini sangat penting karena mengikuti best practice dalam machine learning:

1. `ColumnDropper`: Menghapus kolom identitas yang tidak relevan (ID, `transaction_id`, `user_id`)
2. `DataFrameImputer`: Mengisi nilai hilang—median untuk numerik, modus/"Unknown" untuk kategorikal
3. `FeatureEncoder` (one-hot): Encoding fitur kategorikal dengan sedikit kategori
4. `FeatureEncoder` (target): Encoding fitur kategorikal dengan banyak kategori
5. `FeatureScaler` (log): Transformasi logaritmik untuk fitur sangat skewed
6. `FeatureScaler` (robust): Penskalaan robust untuk seluruh fitur numerik
7. `DimensionalityReducer` (PCA): Reduksi dimensi dengan mempertahankan 95% varian

Pipeline ini di-fit hanya pada data latih (`X_train_raw` dengan `y_train_raw`, karena `y` diperlukan oleh target encoding), lalu hanya di-transform pada data validasi dan test. Ini krusial untuk mencegah data leakage, yaitu bocornya informasi dari data test ke proses training yang bisa membuat evaluasi tidak valid.

Setelah pipeline, SMOTE dijalankan hanya pada data latih untuk menyeimbangkan kelas. Data validasi dan test tetap dibiarkan original agar evaluasi mencerminkan performa model dalam kondisi sesungguhnya.

```
from sklearn.pipeline import Pipeline

# Kolom acuan dari train (aman: nanti difilter terhadap yang tersedia)
```

```

cols_drop = ['ID', 'transaction_id', 'user_id']

# Kategori dengan sedikit nilai unik → One-Hot
cols_onehot = ['gender', 'device_type', 'transaction_type', 'day_of_week',
               'is_new_country']

# Kategori dengan banyak nilai unik → Target/Frequency encoding
cols_target_enc = ['merchant_category', 'country', 'device_os']

# Fitur numerik yang akan dinormalisasi/ditransform
cols_norm = [
    'age', 'transaction_amount', 'time_of_day', 'transaction_duration',
    'num_prev_transactions', 'avg_transaction_amount',
    'std_transaction_amount',
    'transactions_last_24h', 'transactions_last_1h', 'failed_login_attempts',
    'ip_risk_score', 'device_trust_score', 'account_age_days',
    'shared_ip_users', 'shared_device_users', 'merchant_risk', 'country_risk',
    'distance_from_home'
]

# Fitur yang sangat skewed → pakai log1p
cols_log = ['transaction_amount', 'distance_from_home']

# Filter kolom yang benar-benar ada setelah menghapus target & ID
target_col = 'is_fraud'
available_after_drop = [c for c in data_train.columns if c not in set(cols_drop
+ [target_col])]
cols_onehot_safe = [c for c in cols_onehot if c in available_after_drop]
cols_target_enc_safe = [c for c in cols_target_enc if c in
available_after_drop]
cols_norm_safe = [c for c in cols_norm if c in available_after_drop]
cols_log_safe = [c for c in cols_log if c in available_after_drop]

print('[Preprocessing] Kolom yang dipakai:')
print(f" Drop          : {cols_drop}")
print(f" One-Hot       : {cols_onehot_safe}")
print(f" Target Enc    : {cols_target_enc_safe}")
print(f" Log1p         : {cols_log_safe}")

```

```
print(f" RobustScale : {cols_norm_safe}")

# Rangkaian preprocessing tunggal (dipakai untuk train/val/test)
pipe = Pipeline([
    ('dropper', ColumnDropper(cols_to_drop=cols_drop)),
    ('imputer_df', DataFrameImputer()),
    ('encoder_ohe', FeatureEncoder(cols_to_encode=cols_onehot_safe,
encoding_type='onehot')),
    ('encoder_target', FeatureEncoder(cols_to_encode=cols_target_enc_safe,
encoding_type='target')),
    # Log transform untuk fitur sangat skewed
    ('log_amount', FeatureScaler(cols_to_scale=cols_log_safe,
scaler_type='log')),
    # Robust scaler untuk numerik (lebih tahan outlier)
    ('robust_scaler', FeatureScaler(cols_to_scale=cols_norm_safe,
scaler_type='robust')),
    # PCA opsional: jika tidak membantu, set n_components=None
    ('reducer', DimensionalityReducer(n_components=0.95)),
])

print('Pipeline siap dieksekusi.')
```

## BAB II

### Implementasi Decision Tree Learning

Decision Tree adalah algoritma supervised learning non-parametrik yang digunakan untuk klasifikasi dan regresi. Algoritma ini bekerja dengan memecah dataset menjadi subset yang semakin kecil berdasarkan aturan keputusan yang diturunkan dari fitur data, membentuk struktur seperti pohon dengan *root node*, *internal nodes*, dan *leaf nodes*. Implementasi ini berfokus pada algoritma CART (*Classification and Regression Trees*), yang menggunakan *Gini Impurity* sebagai kriteria pemisahan utama untuk menghasilkan pohon biner.

Prinsip utama CART adalah menemukan fitur dan *threshold* terbaik yang meminimalkan *impurity* pada setiap percabangan. Proses ini dilakukan secara rekursif hingga kriteria penghentian terpenuhi, seperti kedalaman maksimum pohon atau jumlah sampel minimum per node.

Berikut adalah langkah-langkah utama dalam penerapan algoritma Decision Tree CART pada implementasi ini:

1. Menentukan *hyperparameter* (membuat pohon) seperti kedalaman maksimum (*max\_depth*) dan jumlah sampel minimum untuk pemisahan (*min\_samples*), serta menginisialisasi struktur pohon.
2. Memilih Split Terbaik dengan cara algoritma mengevaluasi semua fitur dan kemungkinan nilai ambang (*threshold*) untuk menemukan pemisahan yang paling efektif dalam mengurangi *Gini Impurity* (untuk klasifikasi) atau *Mean Squared Error* (untuk regresi).
3. Setelah split terbaik ditemukan, data dipartisi menjadi dua himpunan (kiri dan kanan). Proses ini diulang secara rekursif untuk setiap cabang.
4. Rekursi berhenti jika node sudah murni (hanya berisi satu kelas), kedalaman maksimum tercapai, atau jumlah sampel di node kurang dari batas minimum.
5. Setelah pohon tumbuh penuh, dilakukan *Cost Complexity Pruning* untuk mengurangi kompleksitas pohon dan mencegah *overfitting* dengan menghapus cabang yang memberikan kontribusi minimal terhadap akurasi.
6. Melakukan prediksi dengan menelusuri data dari akar hingga mencapai daun (*leaf node*) berdasarkan aturan keputusan di setiap *node*.

Lalu, berikut kami sediakan implementasi yang kami terapkan untuk Decision Tree CART from scratch:

```
class DecisionTreeCART:

    def __init__(self, max_depth=100, min_samples=2, ccp_alpha=0.0,
regression=False):
        self.max_depth = max_depth
        self.min_samples = min_samples
        self.ccp_alpha = ccp_alpha
        self.regression = regression
        self.tree = None
        self._y_type = None
        self._num_all_samples = None

    def _set_df_type(self, X, y, dtype):
        X = X.astype(dtype)
        y = y.astype(dtype) if self.regression else y
        self._y_dtype = y.dtype

        return X, y

    @staticmethod
    def _purity(y):
        unique_classes = np.unique(y)

        return unique_classes.size == 1

    @staticmethod
    def _is_leaf_node(node):
        return not isinstance(node, dict)

    def _leaf_node(self, y):
        class_index = 0

        return np.mean(y) if self.regression else y.mode()[class_index]
```

```

def _split_df(self, X, y, feature, threshold):
    feature_values = X[feature]
    left_indexes = X[feature_values <= threshold].index
    right_indexes = X[feature_values > threshold].index
    sizes = np.array([left_indexes.size, right_indexes.size])

    return self._leaf_node(y) if any(sizes == 0) else left_indexes,
right_indexes

@staticmethod
def _gini_impurity(y):
    _, counts_classes = np.unique(y, return_counts=True)
    squared_probabilities = np.square(counts_classes / y.size)
    gini_impurity = 1 - sum(squared_probabilities)

    return gini_impurity

@staticmethod
def _mse(y):
    mse = np.mean((y - y.mean()) ** 2)

    return mse

@staticmethod
def _cost_function(left_df, right_df, method):
    total_df_size = left_df.size + right_df.size
    p_left_df = left_df.size / total_df_size
    p_right_df = right_df.size / total_df_size
    J_left = method(left_df)
    J_right = method(right_df)
    J = p_left_df*J_left + p_right_df*J_right

    return J # weighted Gini impurity or weighted mse

def _node_error_rate(self, y, method):
    if self._num_all_samples is None:
        self._num_all_samples = y.size # num samples of all dataframe
    current_num_samples = y.size

```



```

        return current_num_samples / self._num_all_samples * method(y)

    def _best_split(self, X, y):
        features = X.columns
        min_cost_function = np.inf
        best_feature, best_threshold = None, None
        method = self._mse if self.regression else self._gini_impurity

        for feature in features:
            unique_feature_values = np.unique(X[feature])

            for i in range(1, len(unique_feature_values)):
                current_value = unique_feature_values[i]
                previous_value = unique_feature_values[i-1]
                threshold = (current_value + previous_value) / 2
                left_indexes, right_indexes = self._split_df(X, y, feature,
threshold)
                left_labels, right_labels = y.loc[left_indexes],
y.loc[right_indexes]
                current_J = self._cost_function(left_labels, right_labels,
method)

                if current_J <= min_cost_function:
                    min_cost_function = current_J
                    best_feature = feature
                    best_threshold = threshold

        return best_feature, best_threshold

    def _stopping_conditions(self, y, depth, n_samples):
        return self._purity(y), depth == self.max_depth, n_samples <
self.min_samples

    def _grow_tree(self, X, y, depth=0):
        current_num_samples = y.size
        print(f"--> Depth: {depth}, Processing {current_num_samples} samples")
        X, y = self._set_df_type(X, y, np.float64)

```

```

method = self._mse if self.regression else self._gini_impurity

if any(self._stopping_conditions(y, depth, current_num_samples)):
    RTi = self._node_error_rate(y, method) # leaf node error rate
    leaf_node = f'{self._leaf_node(y)} | error_rate {RTi}'
    return leaf_node

Rt = self._node_error_rate(y, method) # decision node error rate
best_feature, best_threshold = self._best_split(X, y)
decision_node = f'{best_feature} <= {best_threshold} | ' \
                f'as_leaf {self._leaf_node(y)} error_rate {Rt}'

left_indexes, right_indexes = self._split_df(X, y, best_feature,
best_threshold)
left_X, right_X = X.loc[left_indexes], X.loc[right_indexes]
left_labels, right_labels = y.loc[left_indexes], y.loc[right_indexes]

# recursive part
tree = {decision_node: []}
left_subtree = self._grow_tree(left_X, left_labels, depth+1)
right_subtree = self._grow_tree(right_X, right_labels, depth+1)

if left_subtree == right_subtree:
    tree = left_subtree
else:
    tree[decision_node].extend([left_subtree, right_subtree])

return tree

def _tree_error_rate_info(self, tree, error_rates_list):
    if self._is_leaf_node(tree):
        _, leaf_error_rate = tree.split()
        error_rates_list.append(np.float128(leaf_error_rate))
    else:
        decision_node = next(iter(tree))
        left_subtree, right_subtree = tree[decision_node]
        self._tree_error_rate_info(left_subtree, error_rates_list)
        self._tree_error_rate_info(right_subtree, error_rates_list)

```

```

RT = sum(error_rates_list) # total leaf error rate of a tree
num_leaf_nodes = len(error_rates_list)

return RT, num_leaf_nodes

@staticmethod
def _ccp_alpha_eff(decision_node_Rt, leaf_nodes_RTt, num_leafs):

    return (decision_node_Rt - leaf_nodes_RTt) / (num_leafs - 1)

def _find_weakest_node(self, tree, weakest_node_info):
    if self._is_leaf_node(tree):
        return tree

    decision_node = next(iter(tree))
    left_subtree, right_subtree = tree[decision_node]
    _, decision_node_error_rate = decision_node.split()

    Rt = np.float128(decision_node_error_rate)
    RTt, num_leaf_nodes = self._tree_error_rate_info(tree, [])
    ccp_alpha = self._ccp_alpha_eff(Rt, RTt, num_leaf_nodes)
    decision_node_index, min_ccp_alpha_index = 0, 1

    if ccp_alpha <= weakest_node_info[min_ccp_alpha_index]:
        weakest_node_info[decision_node_index] = decision_node
        weakest_node_info[min_ccp_alpha_index] = ccp_alpha

    self._find_weakest_node(left_subtree, weakest_node_info)
    self._find_weakest_node(right_subtree, weakest_node_info)

    return weakest_node_info

def _prune_tree(self, tree, weakest_node):
    if self._is_leaf_node(tree):
        return tree

    decision_node = next(iter(tree))

```

```

left_subtree, right_subtree = tree[decision_node]
left_subtree_index, right_subtree_index = 0, 1
_, leaf_node = weakest_node.split('as_leaf ')

if weakest_node is decision_node:
    tree = weakest_node
if weakest_node in left_subtree:
    tree[decision_node][left_subtree_index] = leaf_node
if weakest_node in right_subtree:
    tree[decision_node][right_subtree_index] = leaf_node

self._prune_tree(left_subtree, weakest_node)
self._prune_tree(right_subtree, weakest_node)

return tree

def cost_complexity_pruning_path(self, X: pd.DataFrame, y: pd.Series):
    tree = self._grow_tree(X, y)
    tree_error_rate, _ = self._tree_error_rate_info(tree, [])
    error_rates = [tree_error_rate]
    ccp_alpha_list = [0.0]

    while not self._is_leaf_node(tree):
        initial_node = [None, np.inf]
        weakest_node, ccp_alpha = self._find_weakest_node(tree,
initial_node)
        tree = self._prune_tree(tree, weakest_node)
        tree_error_rate, _ = self._tree_error_rate_info(tree, [])

        error_rates.append(tree_error_rate)
        ccp_alpha_list.append(ccp_alpha)

    return np.array(ccp_alpha_list), np.array(error_rates)

def _ccp_tree_error_rate(self, tree_error_rate, num_leaf_nodes):

    return tree_error_rate + self.ccp_alpha*num_leaf_nodes #
regularization

```

```

def _optimal_tree(self, X, y):
    tree = self._grow_tree(X, y) # grow a full tree
    min_RT_alpha, final_tree = np.inf, None

    while not self._is_leaf_node(tree):
        RT, num_leaf_nodes = self._tree_error_rate_info(tree, [])
        current_RT_alpha = self._ccp_tree_error_rate(RT, num_leaf_nodes)

        if current_RT_alpha <= min_RT_alpha:
            min_RT_alpha = current_RT_alpha
            final_tree = deepcopy(tree)

        initial_node = [None, np.inf]
        weakest_node, _ = self._find_weakest_node(tree, initial_node)
        tree = self._prune_tree(tree, weakest_node)

    return final_tree

def fit(self, X: pd.DataFrame, y: pd.Series):
    self.tree = self._optimal_tree(X, y)

def _traverse_tree(self, sample, tree):
    if self._is_leaf_node(tree):
        leaf, *_ = tree.split()
        return leaf

    decision_node = next(iter(tree)) # dict key
    left_node, right_node = tree[decision_node]
    feature, other = decision_node.split(' <=')
    threshold, *_ = other.split()
    feature_value = sample[feature]

    if np.float128(feature_value) <= np.float128(threshold):
        next_node = self._traverse_tree(sample, left_node) # left_node
    else:
        next_node = self._traverse_tree(sample, right_node) # right_node

```

```

        return next_node

    def predict(self, samples: pd.DataFrame):
        # apply traverse_tree method for each row in a dataframe
        results = samples.apply(self._traverse_tree, args=(self.tree,), axis=1)

        return np.array(results.astype(self._y_dtype))

```

Kelas DecisionTreeCART di atas mengimplementasikan logika pembangunan pohon keputusan secara manual. Metode `_best_split` merupakan inti dari proses pembelajaran, di mana algoritma secara iteratif mengevaluasi setiap nilai fitur untuk menemukan titik potong yang memberikan penurunan *impurity* terbesar. Untuk klasifikasi, digunakan metrik `_gini_impurity`, sedangkan untuk regresi digunakan `_mse`.

Metode `_grow_tree` membangun struktur pohon secara rekursif. Pada setiap langkah, dilakukan pengecekan `_stopping_conditions` (seperti kemurnian node atau kedalaman maksimum). Jika kondisi berhenti terpenuhi, node tersebut menjadi daun (leaf node). Jika tidak, node dipecah menjadi dua cabang baru. Implementasi ini juga menyertakan mekanisme pencetakan log (print) untuk memantau progres kedalaman dan jumlah sampel yang diproses, serta penanganan tipe data `np.float64` untuk kompatibilitas lintas sistem operasi.

Fitur unik dari implementasi ini adalah integrasi *Cost Complexity Pruning* melalui metode `cost_complexity_pruning_path` dan `_optimal_tree`. Setelah pohon tumbuh maksimal, algoritma secara otomatis memangkas cabang-cabang yang tidak efisien berdasarkan parameter `ccp_alpha`, menghasilkan model akhir yang lebih general dan tidak terlalu kompleks.

Kemudian, kami sediakan implementasi yang kami terapkan untuk Decision Tree dengan Scikit-learn:

```

from sklearn.tree import DecisionTreeClassifier
from sklearn.metrics import accuracy_score, classification_report

print("Training DTL (Scikit-Learn)")

```

```

if 'X_train_final' in locals() and 'y_train_final' in locals():
    X_train_dt = X_train_final
    y_train_dt = y_train_final
    X_val_dt = X_val_final
    y_val_dt = y_val_final
else:
    if 'X_train_np' in locals():
        X_train_dt = X_train_np
        y_train_dt = y_train_np
        X_val_dt = X_val_np
        y_val_dt = y_val_np
    else:
        raise ValueError("Data Training tidak ditemukan! Pastikan Section 3
(Preprocessing) sudah dijalankan.")

print(f"Menggunakan data training shape: {X_train_dt.shape}")
print(f"Menggunakan data validasi shape: {X_val_dt.shape}")

# criterion bisa 'entropy' atau 'gini'
dt_model_sk = DecisionTreeClassifier(criterion='entropy', random_state=42)

dt_model_sk.fit(X_train_dt, y_train_dt)

y_pred_dt = dt_model_sk.predict(X_val_dt)
y_prob_dt = dt_model_sk.predict_proba(X_val_dt)[:, 1]

evaluate_model_dtl(y_val_dt, y_pred_dt, y_prob_dt, "Decision Tree
(Scikit-Learn)")

```

Implementasi ini memanfaatkan kelas `DecisionTreeClassifier` dari pustaka `Scikit-learn`, yang menyediakan algoritma Decision Tree yang sangat teroptimasi. Pada implementasi ini, kami menggunakan parameter `criterion` berupa *entropy* yang berarti algoritma menggunakan konsep *Information Gain* (penurunan entropi) untuk menentukan pemisahan terbaik di setiap node. Hal

ini berbeda dengan implementasi *scratch* sebelumnya yang menggunakan *Gini Impurity* (metode CART standar), meskipun keduanya sering memberikan hasil kinerja yang sebanding.

Proses pelatihan dimulai dengan memverifikasi ketersediaan data pelatihan dan validasi. Model kemudian diinisialisasi dengan `random_state=42` untuk memastikan hasil yang konsisten (deterministik). Metode `fit` membangun model pohon keputusan berdasarkan data latih `X_train_dt` dan label `y_train_dt`. Setelah model terbentuk, metode `predict` digunakan untuk mengklasifikasikan data validasi, dan `predict_proba` digunakan untuk mendapatkan skor probabilitas yang diperlukan untuk analisis kurva ROC dan perhitungan AUC. Evaluasi kinerja dilakukan menggunakan fungsi bantu `evaluate_model_dtl` yang menyajikan metrik akurasi, presisi, recall, F1-score, serta *confusion matriks*.



### BAB III

## Implementasi Logistic Regression

Logistic Regression adalah algoritma *supervised learning* yang digunakan untuk tugas klasifikasi, khususnya klasifikasi biner. Algoritma ini memiliki prinsip dasar memodelkan probabilitas kejadian suatu kelas (misalnya 0 atau 1) dengan menggunakan fungsi logistik atau sigmoid untuk memetakan input linear ke dalam rentang nilai antara 0 dan 1 .

Logistic Regression merupakan algoritma yang efisien secara komputasi dan mudah diinterpretasikan. Karakteristik utama dari algoritma ini adalah penggunaan fungsi Sigmoid sebagai fungsi aktivasi yang mengubah keluaran linear ( $Z$ ) menjadi probabilitas. Selain itu, algoritma ini menggunakan *Gradient Descent* untuk meminimalkan *cost function* secara iteratif guna menemukan bobot (*weights*) dan bias yang optimal.

Selain itu, berikut adalah langkah-langkah utama dalam penerapan algoritma Logistic Regression pada implementasi ini:

1. Inisialisasi Parameter: Menentukan *learning rate* dan jumlah iterasi, serta menginisialisasi bobot (*weights*) dan bias awal.
2. Kalkulasi Linear dan Aktivasi: Menghitung kombinasi linear dari fitur input dan bobot, kemudian menerapkannya pada fungsi sigmoid untuk mendapatkan prediksi probabilitas.
3. Perhitungan Cost: Menghitung *error* prediksi menggunakan fungsi *Binary Cross-Entropy* (Log Loss) untuk mengukur seberapa jauh prediksi dari label sebenarnya.
4. Optimasi Gradient Descent: Menghitung gradien (turunan) dari *cost function* terhadap bobot dan bias, kemudian memperbarui parameter tersebut untuk meminimalkan *error*.
5. Prediksi Kelas: Mengonversi probabilitas hasil akhir menjadi label kelas biner (0 atau 1) menggunakan *threshold* tertentu (biasanya 0.5).

Lalu, berikut kami sediakan implementasi yang kami terapkan untuk Logistic Regression *from scratch*:

```
class LogisticRegressionScratch:
    def __init__(self, learning_rate=0.01, iterations=500):
        self.lr = learning_rate
        self.iterations = iterations
        self.weights = None
```

```

self.bias = 0
self.cost_history = []

def sigmoid(self, z):
    return 1 / (1 + np.exp(-z))

def cost(self, h, y):
    m = len(y)
    epsilon = 1e-15
    h = np.clip(h, epsilon, 1 - epsilon)
    return - (1/m) * np.sum(y*np.log(h) + (1-y)*np.log(1-h))

def fit(self, X, y):
    m, n = X.shape
    self.weights = np.zeros(n)
    self.bias = 0

    for i in range(self.iterations):
        z = np.dot(X, self.weights) + self.bias
        h = self.sigmoid(z)

        dw = (1/m) * np.dot(X.T, (h - y))
        db = (1/m) * np.sum(h - y)

        self.weights -= self.lr * dw
        self.bias -= self.lr * db

        if i % 100 == 0:
            self.cost_history.append(self.cost(h, y))

def predict_proba(self, X):
    """ngembaliin nilai probabilitas (0.0 s/d 1.0)"""

```

```
z = np.dot(X, self.weights) + self.bias
return self.sigmoid(z)

def predict(self, X):
    """Ngembaliin nilai hasil (0 atau 1)"""
    return (self.predict_proba(X) >= 0.5).astype(int)
```

Kelas `LogisticRegressionScratch` adalah implementasi algoritma klasifikasi linear yang menggunakan pendekatan optimasi iteratif untuk memisahkan dua kelas data. Algoritma ini bekerja dengan membangun *decision boundary* melalui pembelajaran parameter bobot dan bias.

Pada metode `__init__`, beberapa *hyperparameter* utama diinisialisasi, yaitu `learning_rate` yang mengontrol besarnya langkah penyesuaian bobot selama pelatihan, dan `iterations` yang menentukan jumlah siklus pembelajaran yang akan dilakukan. Variabel `weights`, `bias`, dan `cost_history` juga disiapkan pada tahap ini untuk menyimpan parameter model yang akan diperbarui seiring berjalannya waktu.

Kelas ini dilengkapi dengan metode `sigmoid` yang berfungsi sebagai fungsi aktivasi non-linear, bertugas memetakan nilai input linear ke dalam rentang probabilitas  $[0, 1]$ . Selain itu, terdapat metode `cost` yang menghitung besarnya kesalahan (*loss*) model saat ini menggunakan rumus *Negative Log Likelihood* (Binary Cross Entropy). Metode ini dilengkapi dengan mekanisme *clipping* (`np.clip`) untuk menjaga stabilitas numerik dan mencegah kesalahan perhitungan logaritma pada nilai ekstrem.

Metode `fit` bertugas menjalankan proses pelatihan model (*training*). Di dalamnya, algoritma melakukan iterasi sebanyak jumlah yang ditentukan untuk menghitung prediksi maju (*forward propagation*), menghitung gradien (`dw` dan `db`) berdasarkan selisih antara prediksi (*hypothesis*) dengan label aktual, serta memperbarui parameter `weights` dan `bias` menggunakan algoritma *Gradient Descent*. Nilai *cost* disimpan secara berkala untuk memantau konvergensi model.

Setelah model dilatih, metode `predict_proba` digunakan untuk menghasilkan nilai probabilitas murni berdasarkan input fitur baru. Kemudian, metode `predict` memanfaatkan probabilitas

tersebut untuk menentukan label kelas akhir (0 atau 1) dengan menerapkan *threshold* keputusan sebesar 0.5.

Kemudian, kami sediakan implementasi yang kami terapkan untuk Logistic Regression dengan Scikit-learn:

```
from sklearn.linear_model import LogisticRegression
from sklearn.metrics import accuracy_score, classification_report

# data Training
if 'X_train_np' in locals() and 'y_train_np' in locals():
    X_train_fix = X_train_np
    y_train_fix = y_train_np
elif 'X_train_res' in locals() and 'y_train_res' in locals():
    X_train_fix = X_train_res
    y_train_fix = y_train_res
else:
    raise ValueError("Data Training tidak ditemukan! Pastikan Anda sudah menjalankan Cell 'Persiapan Data' & 'SMOTE' sebelumnya.")

# data Validasi
if 'X_val_np' in locals() and 'y_val_np' in locals():
    X_test_fix = X_val_np
    y_test_fix = y_val_np
elif 'X_val_split' in locals() and 'y_val_split' in locals():
    X_test_fix = X_val_split
    y_test_fix = y_val_split
else:
    raise ValueError("Data Validasi tidak ditemukan! Pastikan cell Split Data sudah dijalankan.")

print(f"Menggunakan data training shape: {X_train_fix.shape}")
print(f"Menggunakan data validasi shape: {X_test_fix.shape}")
```

```

model_sk = LogisticRegression(max_iter=1000, random_state=42)

# Training
model_sk.fit(X_train_fix, y_train_fix)

# Prediksi data validasi
y_pred_sk = model_sk.predict(X_test_fix)

# Eval
accuracy = accuracy_score(y_test_fix, y_pred_sk)
print(f"\nAkurasi (Scikit-learn): {accuracy * 100:.2f}%")
print("\nReport Klasifikasi:")
print(classification_report(y_test_fix, y_pred_sk))

# Prediksi data validasi
y_pred_sk = model_sk.predict(X_test_fix)

# Eval
accuracy = accuracy_score(y_test_fix, y_pred_sk)
print(f"\nAkurasi (Scikit-learn): {accuracy * 100:.2f}%")
print("\nReport Klasifikasi:")
print(classification_report(y_test_fix, y_pred_sk))

```

Implementasi ini memanfaatkan kelas `LogisticRegression` dari pustaka `Scikit-learn`, yang menyediakan solusi teroptimasi dan efisien untuk tugas klasifikasi linear tanpa perlu mendefinisikan perhitungan matematis secara manual.

Pada tahap awal, dilakukan mekanisme pemeriksaan data yang fleksibel untuk menetapkan variabel pelatihan (`X_train_fix`, `y_train_fix`) dan validasi (`X_test_fix`, `y_test_fix`). Logika kondisional ini memastikan bahwa model menerima input yang valid, baik dari data *numpy array* biasa maupun data yang telah melalui proses *resampling* (seperti SMOTE), serta memberikan

notifikasi kesalahan (*raise ValueError*) jika data yang dibutuhkan tidak tersedia dalam *environment*.

Inisialisasi model dilakukan dengan mengatur parameter `max_iter` sebesar 1000. Pengaturan ini bertujuan memberikan kesempatan iterasi yang cukup bagi algoritma *solver* internal untuk mencapai konvergensi, terutama pada dataset yang kompleks. Selain itu, `random_state` ditetapkan pada nilai 42 untuk menjamin *reproducibility*, memastikan bahwa hasil pelatihan tetap konsisten setiap kali kode dijalankan.

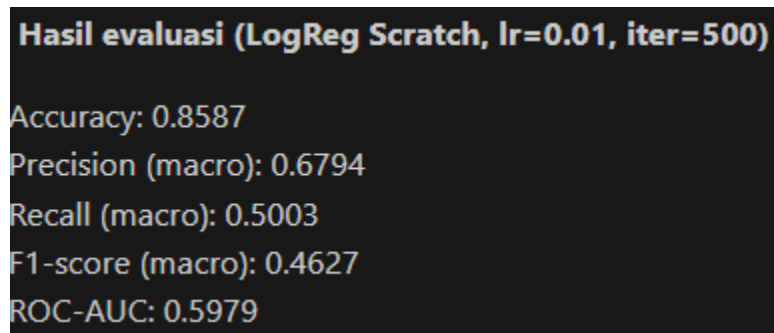
Metode `fit` bertugas melatih model dengan mempelajari pola hubungan antara fitur dan label pada data pelatihan. Berbeda dengan implementasi *from scratch*, metode ini menggunakan algoritma optimasi tingkat lanjut (seperti LBFGS atau LIBLINEAR) yang terintegrasi di dalam *library*. Setelah latihan selesai, metode `predict` digunakan untuk mengklasifikasikan data validasi. Kinerja model kemudian dievaluasi secara komprehensif menggunakan `accuracy_score` untuk melihat akurasi global, serta `classification_report` yang menyajikan metrik *precision*, *recall*, dan *f1-score* untuk analisis performa per kelas.

## **BAB IV**

### **Perbandingan Hasil**

#### **4.1. Analisis Logistic Regression**

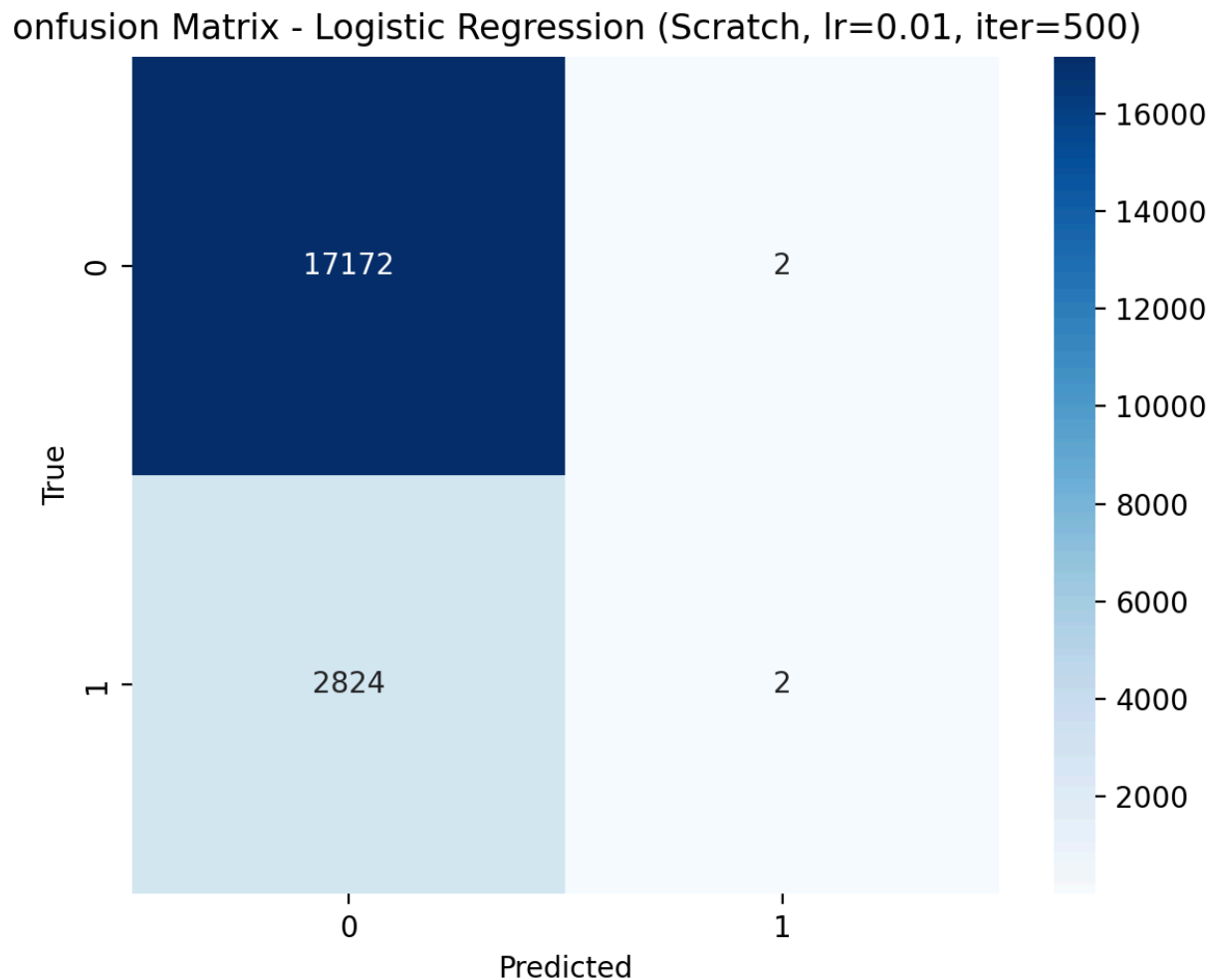
Pada bagian ini dilakukan evaluasi dan analisis performa model Logistic Regression untuk klasifikasi fraud. Evaluasi dilakukan pada data validasi dengan metrik utama Accuracy, Precision (macro), Recall (macro), F1-score (macro), dan ROC-AUC. Selain metrik numerik, kami juga menggunakan confusion matrix untuk melihat pola kesalahan prediksi model.



The image shows a terminal window with a black background and yellow text. The title is 'Hasil evaluasi (LogReg Scratch, lr=0.01, iter=500)'. Below the title, five performance metrics are listed: Accuracy: 0.8587, Precision (macro): 0.6794, Recall (macro): 0.5003, F1-score (macro): 0.4627, and ROC-AUC: 0.5979.

```
Hasil evaluasi (LogReg Scratch, lr=0.01, iter=500)
Accuracy: 0.8587
Precision (macro): 0.6794
Recall (macro): 0.5003
F1-score (macro): 0.4627
ROC-AUC: 0.5979
```

Gambar 4.1 Hasil Perbandingan performa LogReg Scratch



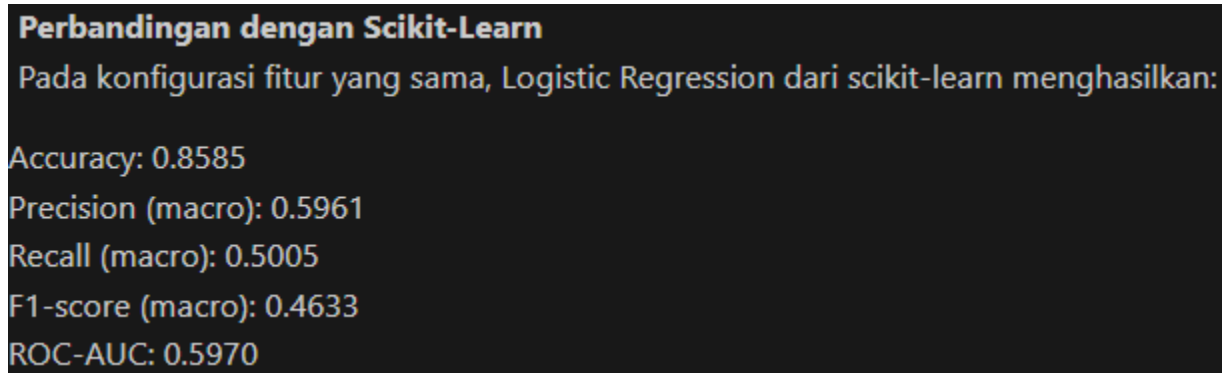
Gambar 4.2 Hasil Perbandingan performa LogReg Scratch

Selanjutnya, confusion matrix untuk model scratch (lihat Gambar 4.2 – Hasil Perbandingan performa LogReg Scratch) menunjukkan distribusi prediksi benar dan salah pada masing-masing kelas. Dari confusion matrix ini dapat diamati bahwa model cenderung lebih stabil pada kelas mayoritas (non-fraud), sedangkan pada kelas minoritas (fraud) performa masih terbatas, yang tercermin dari nilai Recall (macro) dan F1-score (macro) yang relatif lebih rendah.

Catatan penting: karena dataset bersifat imbalanced, nilai Accuracy yang tinggi belum tentu menunjukkan performa yang baik untuk mendeteksi fraud. Oleh sebab itu, metrik seperti



F1-score (macro) dan ROC-AUC digunakan sebagai indikator tambahan untuk menilai kemampuan model dalam membedakan kedua kelas secara lebih adil.

A screenshot of a terminal window with a dark background and light-colored text. The text displays performance metrics for a Logistic Regression model. The title is 'Perbandingan dengan Scikit-Learn'. Below it, a line of text states 'Pada konfigurasi fitur yang sama, Logistic Regression dari scikit-learn menghasilkan:'. The metrics listed are: Accuracy: 0.8585, Precision (macro): 0.5961, Recall (macro): 0.5005, F1-score (macro): 0.4633, and ROC-AUC: 0.5970.

```
Perbandingan dengan Scikit-Learn  
Pada konfigurasi fitur yang sama, Logistic Regression dari scikit-learn menghasilkan:  
Accuracy: 0.8585  
Precision (macro): 0.5961  
Recall (macro): 0.5005  
F1-score (macro): 0.4633  
ROC-AUC: 0.5970
```

Gambar 4.3 Hasil Perbandingan performa LogReg Pustaka

Hasil ini menunjukkan bahwa secara umum performa model scikit-learn berada pada kisaran yang sangat mirip dengan implementasi scratch, terutama pada metrik ROC-AUC (sekitar 0.597) dan F1-score (macro) (sekitar 0.463). Kemiripan hasil ini mengindikasikan bahwa implementasi scratch telah berjalan dengan benar dan menghasilkan perilaku model yang sebanding dengan implementasi library.

Dari kedua percobaan, dapat disimpulkan bahwa:

- Model Logistic Regression (scratch maupun scikit-learn) memberikan performa yang cukup konsisten satu sama lain.
- Nilai ROC-AUC  $\sim 0.597$  menunjukkan kemampuan pemisahan kelas yang masih moderat, sehingga masih terdapat ruang perbaikan melalui tuning lebih lanjut atau feature engineering.
- Pada kasus fraud detection yang imbalanced, evaluasi tidak cukup hanya mengandalkan accuracy; perlu memperhatikan metrik berbasis keseimbangan kelas seperti F1-score (macro) dan ROC-AUC, serta melihat pola error pada confusion matrix.

## 4.2. Analisis Decision Tree Learning

Perbandingan antara implementasi *Decision Tree* dari *scratch* dan yang menggunakan *library* Scikit-learn menunjukkan perbedaan performa yang cukup signifikan, baik dari segi akurasi maupun karakteristik prediksi model. Hal ini dipengaruhi oleh perbedaan mendasar dalam volume data latih yang digunakan serta optimasi algoritma.

Pada implementasi *from scratch*, pelatihan dilakukan menggunakan subset data sebanyak 10.000 sampel yang dipilih secara acak, berbeda dengan implementasi Scikit-learn yang menggunakan seluruh dataset. Keputusan ini diambil demi menjaga efisiensi waktu komputasi. Mengingat implementasi *scratch* dibangun menggunakan struktur data Python murni tanpa optimasi tingkat rendah (seperti C/Cython yang digunakan Scikit-learn), proses pembangunan pohon (*tree growing*) dan perhitungan *impurity* pada dataset berukuran penuh membutuhkan waktu yang eksponensial dan tidak efisien untuk diselesaikan dalam batas waktu pengerjaan.

Secara kuantitatif, implementasi *from scratch* menghasilkan akurasi sebesar 54.34%, sedangkan implementasi menggunakan Scikit-learn mencatatkan akurasi yang lebih tinggi, yaitu 68.16%. Selisih akurasi sekitar 14% ini wajar terjadi karena model Scikit-learn memiliki akses ke informasi pola data yang jauh lebih lengkap (lebih banyak sampel latih) dibandingkan model *scratch* yang hanya melihat sebagian kecil dari populasi data.

```

Evaluasi: Decision Tree (Scratch - 10000 Random Data)
Model Evaluation Metrics untuk Decision Tree (Scratch - 10000 Random Data)
Accuracy          : 0.5434
Precision (Macro) : 0.5096
Recall (Macro)    : 0.5195
F1 Score (Macro)  : 0.4533
ROC AUC Score     : 0.5195

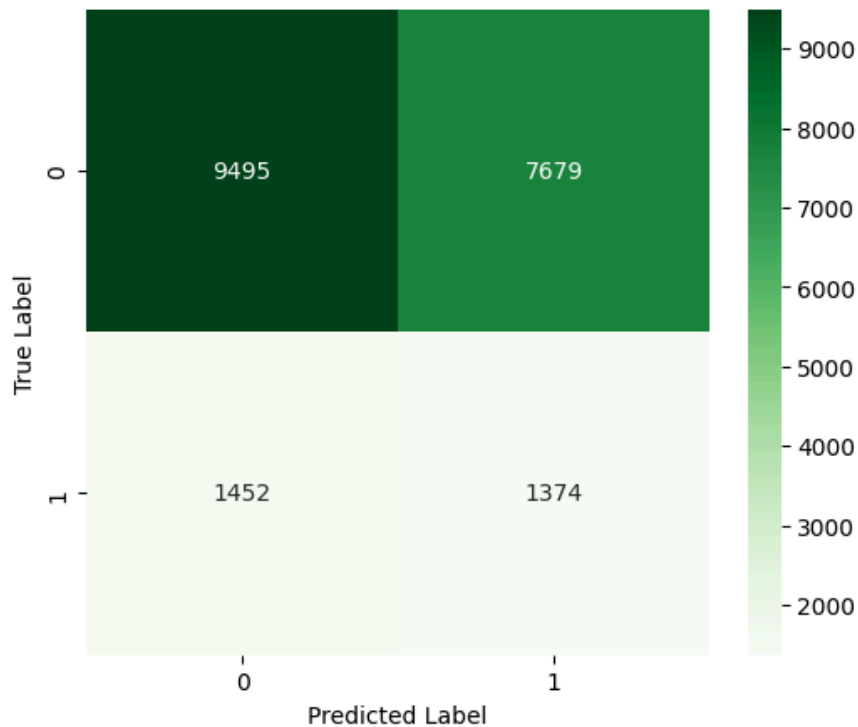
report detail

```

|              | precision | recall | f1-score | support |
|--------------|-----------|--------|----------|---------|
| 0            | 0.87      | 0.55   | 0.68     | 17174   |
| 1            | 0.15      | 0.49   | 0.23     | 2826    |
| accuracy     |           |        | 0.54     | 20000   |
| macro avg    | 0.51      | 0.52   | 0.45     | 20000   |
| weighted avg | 0.77      | 0.54   | 0.61     | 20000   |

Gambar 4.4 Hasil performa DTL Scratch

Confusion Matrix - Decision Tree (Scratch - 10000 Random Data)



Gambar 4.5 Hasil performa DTL Scratch

Meskipun akurasi lebih rendah, model *scratch* justru memiliki nilai Recall untuk kelas minoritas (Fraud/1) sebesar 0.49, yang lebih tinggi dibandingkan model Scikit-learn. Artinya, model ini mampu mendeteksi hampir separuh dari total kasus penipuan. Namun, hal ini harus dibayar mahal dengan tingginya angka *False Positive* (7.679 kejadian), yang menyebabkan nilai Precision sangat rendah (0.15). Ini mengindikasikan bahwa model *scratch* cenderung "agresif" dan *over-sensitive* dalam memprediksi label *fraud* karena keterbatasan data latih yang membuatnya sulit membentuk keputusan yang presisi.

```

Training DTL (Scikit-Learn)
Menggunakan data training shape: (137396, 21)
Menggunakan data validasi shape: (20000, 21)

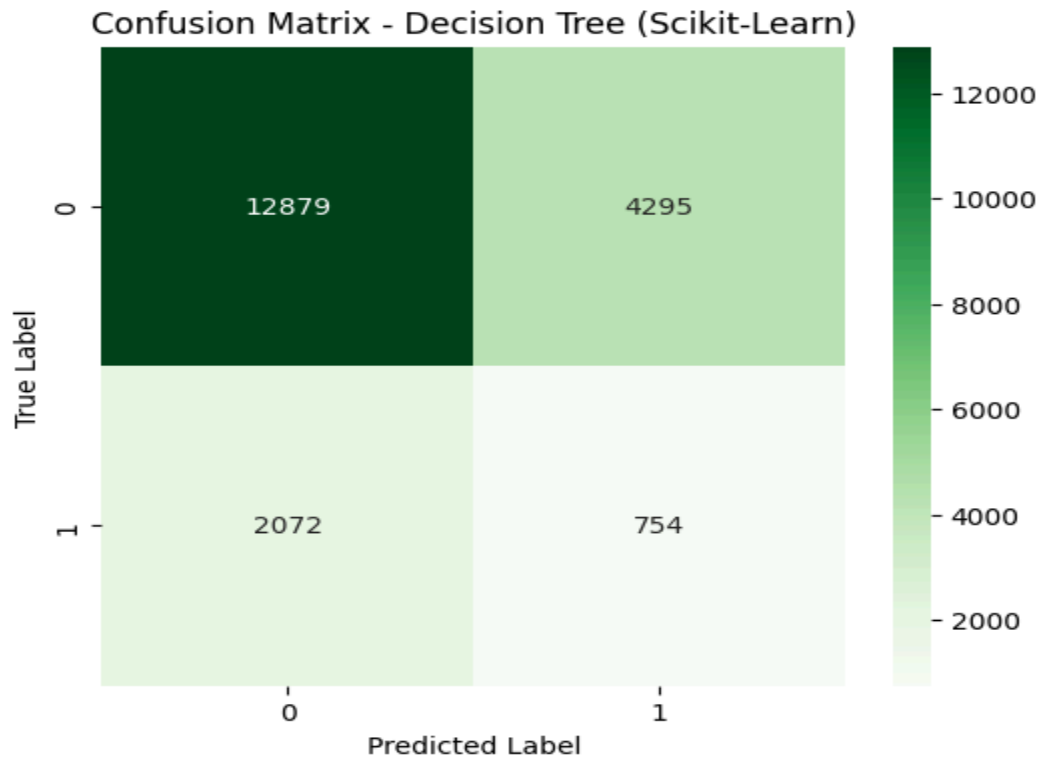
Evaluasi: Decision Tree (Scikit-Learn)
Model Evaluation Metrics untuk Decision Tree (Scikit-Learn)
Accuracy          : 0.6816
Precision (Macro)  : 0.5054
Recall (Macro)     : 0.5084
F1 Score (Macro)   : 0.4966
ROC AUC Score      : 0.5084

report detail

```

|              | precision | recall | f1-score | support |
|--------------|-----------|--------|----------|---------|
| 0            | 0.86      | 0.75   | 0.80     | 17174   |
| 1            | 0.15      | 0.27   | 0.19     | 2826    |
| accuracy     |           |        | 0.68     | 20000   |
| macro avg    | 0.51      | 0.51   | 0.50     | 20000   |
| weighted avg | 0.76      | 0.68   | 0.72     | 20000   |

Gambar 4.5 Hasil performa DTL Pustaka



Gambar 4.6 Hasil performa DTL Pustaka

Model *library* menunjukkan perilaku yang lebih konservatif. Terlihat akurasi globalnya lebih baik (68.16%) dengan jumlah *False Positive* yang lebih sedikit (4.295), model ini memiliki **Recall kelas minoritas yang sangat rendah (0.27)**. Artinya, model ini gagal mendeteksi mayoritas kasus penipuan (banyak *False Negative* sebesar 2.072 kasus). Rendahnya *Recall* ini menunjukkan bahwa meskipun model lebih stabil terhadap *noise*, ia cenderung bias ke arah kelas mayoritas (Normal atau kelas 0).

Secara keseluruhan, kedua model masih menghadapi tantangan dalam menangani ketidakseimbangan kelas (*imbalanced data*), yang terlihat dari rendahnya F1-Score untuk kelas 1 pada kedua implementasi (0.23 untuk *scratch* dan 0.19 untuk Scikit-learn). Penggunaan SMOTE tampaknya belum cukup untuk membantu model *Decision Tree* memisahkan area *fraud* dan *normal* yang saling bertumpuk (*overlapping*) secara efektif pada dataset ini dan perlu dibantu oleh optimisasi yang lain.

## BAB V

### Akhir & Referensi

#### 5.1 Pembagian Tugas

Berikut adalah pembagian tugas setiap anggota kelompok:

| No | NIM      | Nama                        | Deskripsi                                                                                                                                                                                                                                           |
|----|----------|-----------------------------|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| 1. | 18223004 | Muhammad Farhan             | <ul style="list-style-type: none"><li>- Mengerjakan Algoritma/implementasi DTL + Logistic Regression</li><li>- Mengerjakan Perbandingan hasil Algoritma DTL + Logistic Regression</li><li>- <i>Submit</i> Kaggle</li></ul>                          |
| 2. | 18223026 | Jacob Reinhard M. Siagian   | <ul style="list-style-type: none"><li>- Mengerjakan Algoritma/implementasi Logistic Regression</li><li>- Mengerjakan Perbandingan hasil Algoritma DTL + Logistic Regression</li><li>- <i>Submit</i> Kaggle</li><li>- Mengerjakann Laporan</li></ul> |
| 3. | 18223028 | Stevan Einer Bonagabe       | <ul style="list-style-type: none"><li>- Mengerjakan Code bagian Data Cleaning</li><li>- Mengerjakann Laporan</li></ul>                                                                                                                              |
| 4. | 18223072 | Hans Joseph B. W. Silitonga | <ul style="list-style-type: none"><li>- Mengerjakan Code bagian Data Preprocessing dan Preprocessing Pipeline</li><li>- Mengerjakann Laporan</li><li>- Melakukan Submit pada Kaggle</li></ul>                                                       |

#### 5.2 Referensi

1. Scikit-learn Documentation: <https://scikit-learn.org/>
2. Imbalanced-learn (SMOTE): <https://imbalanced-learn.org/>
3. Chawla, N. V., et al. (2002). "SMOTE: Synthetic Minority Over-sampling Technique"

4. Hastie, T., Tibshirani, R., & Friedman, J. (2009). "The Elements of Statistical Learning"